OAVS EXAM PREPARATION SERIES

COMPUTER TEACHER (TGT)
C PROGRAMMING

Constants in C

Complete MCQ Workbook

120 Practice Questions with Explanations

- Integer Constants
- Floating-Point Constants
- Character Constants
- String Constants
- Symbolic Constants

Includes: Syntax Questions • Error-Finding • Answer Key • Detailed Explanations

For OAVS, KVS, NVS, DSSSB & State TGT/PGT Exams

HOW TO USE THIS BOOK

Structure	This book contains 120 MCQs divided into 5 sections covering all subtopics of Constants in C.
Attempt First	Try to answer each question independently before reading the explanation.
Answer Key	A consolidated answer key is provided at the end of the book for quick reference.
Explanations	Detailed explanations follow each question to strengthen conceptual understanding.
Error-Finding	Special error-finding questions are marked and test your ability to spot C syntax mistakes.
Exam Strategy	Each question is exam-pattern aligned with OAVS/KVS/NVS TGT Computer Teacher exams.

SECTION OVERVIEW

Section	Topic	Questions	Key Concepts
1	Integer Constants	Q 1–25	Decimal, Octal, Hex, Suffixes
2	Floating-Point Constants	Q 26–50	float, double, Exponents, Suffixes
3	Character Constants	Q 51–75	ASCII, Escape Sequences, Type
4	String Constants	Q 76–100	Null-terminator, strlen, Literals
5	Symbolic Constants	Q 101–120	#define, const, Macros, Scope

SECTION — INTEGER CONSTANTS

Q1.

Which of the following is a valid integer constant in C?

(A) 123 ✓

(B) 12.3

(C) 'A'

(D) "123"

Explanation: *123 is a plain decimal integer constant. 12.3 is a floating-point constant, 'A' is a character constant, and "123" is a string literal — none of them are integer constants.*

Q2.

What is the octal representation of the decimal number 8 in C?

(A) 08

(B) 010 ✓

(C) 0x8

(D) 8O

Explanation: *Octal constants in C are prefixed with 0 (zero). Decimal 8 = Octal 10, so it is written as 010. Note: 08 is invalid because 8 is not a valid octal digit.*

Q3.

Which prefix is used for hexadecimal constants in C?

(A) 0b

(B) 0o

(C) 0x or 0X ✓

(D) H

Explanation: *Hexadecimal constants in C are prefixed with 0x or 0X (case-insensitive). For example, 0xFF represents 255 in decimal.*

Q4.

What is the value of the hexadecimal constant 0x1A in decimal?

(A) 10

(B) 16

(C) 26 ✓

(D) 31

Explanation: *0x1A = 1×16 + 10 = 16 + 10 = 26. In hexadecimal, A = 10.*

Q5.

Which of the following is NOT a valid integer constant in C?

(A) 0777

(B) 0xFF

(C) 09 ✓

(D) 2147483647

Explanation: *09 is invalid because octal numbers use digits 0–7 only. The digit 9 is not a valid octal digit, making 09 a syntax error.*

Q6.

What suffix is used to declare an unsigned long integer constant?

(A) L

(B) U

(C) UL ✓

(D) ULL

Explanation: *The suffix UL (or ul) denotes an unsigned long integer constant. U alone means unsigned int, L alone means long int, and ULL means unsigned long long int.*

Q7.

Which of the following integer constants has the largest value?

(A) 0377

(B) 0xFF ✓

(C) 255

(D) 0b11111111

Explanation: *0377 (octal) = 255, 0xFF (hex) = 255, 255 (decimal) = 255. All represent the same value 255. 0b... is not standard C (it is a GCC extension). All equal values here — 0xFF is listed first among equals.*

Q8.

What is the decimal equivalent of the octal constant 0755?

(A) 493 ✓

(B) 755

(C) 509

(D) 461

Explanation: *0755 (octal) = 7×64 + 5×8 + 5×1 = 448 + 40 + 5 = 493.*

Q9.

Which statement about integer constants is CORRECT?

(A) Octal constants start with 0x

(B) Hexadecimal constants start with 0

(C) Integer constants cannot be negative

(D) A leading zero makes a constant octal ✓

Explanation: *A leading zero (0) makes an integer constant octal. Hex constants start with 0x or 0X. Negative integers use the unary minus operator applied to a positive constant.*

Q10.

What is the output of: printf("%d", 010 + 0x10);?

(A) 20

(B) 26

(C) 24 ✓

(D) 18

Explanation: *010 is octal 10 = decimal 8; 0x10 is hex 10 = decimal 16. Sum = 8 + 16 = 24.*

Q11.

The suffix 'LL' after an integer constant indicates:

(A) Long literal

(B) Long long integer ✓

(C) Linked list

(D) Large limit

Explanation: *The suffix LL (or ll) denotes a long long integer constant, which is at least 64 bits wide. Example: 9223372036854775807LL.*

Q12.

Which of the following correctly represents the integer constant 255 in C?

(A) 0xFF

(B) 0377

(C) 255

(D) All of the above ✓

Explanation: *All three — 0xFF (hex), 0377 (octal), and 255 (decimal) — represent the same integer value 255 in C. They are interchangeable.*

Q13.

What error does the following cause? int x = 09;

(A) No error; 09 is treated as decimal

(B) Warning: implicit conversion

(C) Compile error: invalid octal digit ✓

(D) Runtime error: overflow

Explanation: *09 is interpreted as an octal literal (leading zero), but 9 is not a valid octal digit (0–7). This causes a compile-time error.*

Q14.

What is the range of a signed integer constant on a 32-bit system?

(A) 0 to 65535

(B) -2147483648 to 2147483647 ✓

(C) -32768 to 32767

(D) 0 to 4294967295

Explanation: *A signed 32-bit integer (int) ranges from -2^31 = -2147483648 to 2^31-1 = 2147483647.*

Q15.

Which is a valid long integer constant in C?

(A) 123L

(B) 123l

(C) 123UL

(D) All of the above ✓

Explanation: *123L and 123l both denote a long integer constant. 123UL denotes an unsigned long integer constant. All three are valid in C.*

Q16.

How many digits can a hexadecimal constant use?

(A) 0-9 only

(B) 0-9 and A-F (or a-f) ✓

(C) 0-7 only

(D) 0-9 and A-Z

Explanation: *Hexadecimal uses base-16 digits: 0–9 and A–F (or a–f). Letters A–F represent values 10–15.*

Q17.

What is the output of printf("%o", 8);?

(A) 8

(B) 010

(C) 10 ✓

(D) 0x8

Explanation: *%o prints an integer in octal (without the 0 prefix). Decimal 8 = octal 10, so the output is 10.*

Q18.

Which of the following constants, if stored in a signed char, will cause overflow?

(A) 100

(B) 127

(C) 128 ✓

(D) -128

Explanation: *A signed char holds values from -128 to 127. Storing 128 exceeds the maximum value, causing overflow (undefined behaviour).*

Q19.

What does the U suffix on an integer constant indicate?

(A) Unicode

(B) Unsigned ✓

(C) Unary

(D) Utility

Explanation: *The U (or u) suffix marks an integer constant as unsigned, e.g., 300U is an unsigned int constant.*

Q20.

Which of the following is a valid integer constant in C? (Error-finding)

(A) 1,000

(B) 1_000

(C) 1000 ✓

(D) 1.000

Explanation: *In C, integer constants cannot contain commas, underscores (C17 and earlier), or decimal points. Only 1000 is valid. Underscores as digit separators are not part of standard C (they exist in C++ 14 and later).*

Q21.

What is the value of 0xA + 011?

(A) 19 ✓

(B) 21

(C) 11

(D) 17

Explanation: *0xA = 10 (hex); 011 = 9 (octal). Sum = 10 + 9 = 19.*

Q22.

Which format specifier prints an integer in hexadecimal (lowercase)?

(A) %o

(B) %x ✓

(C) %X

(D) %h

Explanation: *%x prints an unsigned integer in lowercase hexadecimal. %X uses uppercase. %o prints in octal.*

Q23.

Which of the following best describes a constant in C?

(A) A variable whose value can change

(B) A fixed value that does not change during program execution ✓

(C) A function that returns a fixed value

(D) A pointer to a fixed memory location

Explanation: *A constant is a fixed, literal value used directly in the source code. It does not change during program execution.*

Q24.

What is the type of the constant 2147483648 on a 32-bit int system?

(A) int

(B) unsigned int

(C) long int ✓

(D) Depends on the compiler

Explanation: *2147483648 exceeds INT_MAX (2147483647) for a 32-bit int. The compiler automatically promotes it to long int (or long long int), per C standard rules.*

Q25.

What will int x = 0x10 + 010 + 10; store in x?

(A) 30

(B) 34 ✓

(C) 36

(D) 38

Explanation: *0x10 = 16, 010 = 8, 10 = 10. Sum = 16 + 8 + 10 = 34.*

SECTION — FLOATING-POINT CONSTANTS

Q26.

Which of the following is a valid floating-point constant in C?

(A) 3.14

(B) 3.

(C) .14

(D) All of the above ✓

Explanation: *All three are valid floating-point constants. 3.14 is the most common form. 3. (with trailing dot) equals 3.0, and .14 (with leading dot) equals 0.14.*

Q27.

What is the default type of a floating-point constant like 3.14 in C?

(A) float

(B) double ✓

(C) long double

(D) Depends on value

Explanation: *By default, a floating-point literal like 3.14 is of type double in C. Use the f suffix for float (3.14f) and l/L for long double (3.14L).*

Q28.

Which suffix makes a floating-point constant of type float?

(A) d or D

(B) f or F ✓

(C) l or L

(D) e or E

Explanation: *The f or F suffix designates a float constant (e.g., 3.14f). Without a suffix it is double; with l or L it is long double.*

Q29.

What does the scientific notation 1.5e3 represent?

(A) 0.0015

(B) 15

(C) 1500.0 ✓

(D) 1.5003

Explanation: *1.5e3 means 1.5 × 10^3 = 1500.0. The 'e' separates the mantissa from the exponent in scientific (exponential) notation.*

Q30.

Which of the following is NOT a valid floating-point constant?

(A) 1e5

(B) 1.0e-5

(C) 1.5E+3

(D) 1e ✓

Explanation: *1e requires an exponent digit after 'e'. Without at least one digit for the exponent, the constant is invalid. 1e5, 1.0e-5, and 1.5E+3 are all valid.*

Q31.

What type does the constant 3.14L represent?

(A) float

(B) double

(C) long double ✓

(D) long float

Explanation: *The L (or l) suffix denotes a long double constant. Note: 'long float' is not a valid type in C.*

Q32.

What is the approximate range of a double in C?

(A) ±1.2×10^-38 to ±3.4×10^38

(B) ±2.2×10^-308 to ±1.8×10^308 ✓

(C) ±3.4×10^-4932 to ±1.1×10^4932

(D) ±1.0×10^-10 to ±1.0×10^10

Explanation: *A double (64-bit IEEE 754) has approximately 15–16 significant digits and an exponent range of ±2.2×10^-308 to ±1.8×10^308.*

Q33.

What is the value of 2.5e-2?

(A) 250.0

(B) 25.0

(C) 0.025 ✓

(D) 0.0025

Explanation: *2.5e-2 = 2.5 × 10^-2 = 2.5 / 100 = 0.025.*

Q34.

What format specifier is used to print a float/double with printf?

(A) %i

(B) %f ✓

(C) %c

(D) %s

Explanation: *%f prints a floating-point number in decimal notation. %e or %E prints in scientific notation, and %g selects the shorter of the two.*

Q35.

What is the error in: float x = 3.14;?

(A) No error ✓

(B) Implicit truncation: 3.14 is double but stored in float

(C) Syntax error: missing f suffix

(D) Runtime error: precision lost

Explanation: *There is no error. C allows implicit conversion of double to float. A warning may appear with strict settings, but it compiles correctly.*

Q36.

Which is the correct way to declare a float constant 3.14?

(A) 3.14

(B) 3.14F

(C) 3.14f

(D) Both B and C ✓

Explanation: *Both 3.14F and 3.14f are valid float constants. The f/F suffix forces the type to float instead of the default double.*

Q37.

How many bytes does a float typically occupy?

(A) 2

(B) 4 ✓

(C) 8

(D) 16

Explanation: *A float is typically 4 bytes (32 bits) on most platforms, following the IEEE 754 single-precision format.*

Q38.

What is the output of printf("%f", 1/2);?

(A) 0.500000

(B) 0.000000 ✓

(C) 0

(D) Error

Explanation: *1/2 is integer division (both operands are int), yielding 0 (int). %f then prints 0 as 0.000000 due to type mismatch (undefined behaviour, but typically prints 0.000000).*

Q39.

Which of the following represents negative infinity as a floating-point concept?

(A) -1e999 ✓

(B) INT_MIN

(C) -DBL_MAX - 1

(D) None can represent it exactly

Explanation: *In practice, -1e999 overflows to -INFINITY in IEEE 754. The macro -INFINITY (from math.h) is the correct way, but -1e999 is commonly used to demonstrate the concept.*

Q40.

What is the precision (significant decimal digits) of a float?

(A) 6–7 ✓

(B) 15–16

(C) 18–19

(D) 3–4

Explanation: *A 32-bit float has approximately 6–7 significant decimal digits of precision. A double has 15–16.*

Q41.

Which of the following correctly identifies a syntax error?

(A) double x = .5;

(B) double y = 5.;

(C) double z = 5e; ✓

(D) double w = 5E-2;

Explanation: *5e is invalid because the exponent 'e' must be followed by at least one digit. The other forms (.5, 5., and 5E-2) are valid floating-point constants.*

Q42.

What does printf("%.2f", 3.14159); output?

(A) 3.14159

(B) 3.14 ✓

(C) 3.1

(D) 3.00

Explanation: *The format %.2f specifies 2 decimal places. 3.14159 rounded to 2 places is 3.14.*

Q43.

What is the long double suffix?

(A) ld or LD

(B) l or L ✓

(C) lf or LF

(D) LL

Explanation: *The l or L suffix on a floating-point literal makes it long double, e.g., 3.14159265358979L.*

Q44.

What does the 'e' or 'E' in a floating-point constant denote?

(A) Error

(B) Exponent (power of 10) ✓

(C) Extension

(D) Enumeration

Explanation: *In scientific notation, 'e' or 'E' separates the mantissa from the exponent (power of 10). E.g., 1.5e3 = 1.5 × 10^3.*

Q45.

What is the value of 0.1 + 0.2 == 0.3 in C?

(A) 1 (true)

(B) 0 (false) ✓

(C) Undefined

(D) Depends on OS

Explanation: *Due to floating-point representation errors, 0.1 + 0.2 gives approximately 0.30000000000000004, which is not exactly equal to 0.3. The expression evaluates to 0 (false).*

Q46.

What is the correct format specifier to print a long double?

(A) %f

(B) %lf

(C) %Lf ✓

(D) %ld

Explanation: *%Lf is the correct format specifier for long double. %lf is for double in scanf, and %f is for float/double in printf.*

Q47.

Which of the following is equivalent to 1.5E+3?

(A) 0.0015

(B) 1500.0 ✓

(C) 15.0

(D) 150.0

Explanation: *1.5E+3 = 1.5 × 10^3 = 1500.0.*

Q48.

Which header file defines DBL_MAX and FLT_MAX?

(A) limits.h

(B) float.h ✓

(C) stdlib.h

(D) math.h

Explanation: *float.h defines constants like FLT_MAX, DBL_MAX, FLT_MIN, DBL_MIN that describe floating-point type limits.*

Q49.

Error-finding: What is wrong with double d = 3,14;?

(A) No error

(B) Comma is not a valid decimal separator in C ✓

(C) double cannot hold 3.14

(D) Missing semicolon

Explanation: *In C, the decimal point must be a period (.). A comma is the comma operator, not a decimal separator. 3,14 is parsed as expression 3 followed by 14, causing a syntax error.*

Q50.

What is the size of a double in bytes (typically)?

(A) 4

(B) 8 ✓

(C) 10

(D) 16

Explanation: *A double is typically 8 bytes (64 bits), following the IEEE 754 double-precision standard.*

SECTION — CHARACTER CONSTANTS

Q51.

How is a character constant written in C?

(A) "A"

(B) 'A' ✓

(C) A

(D) \A

Explanation: *Character constants in C are enclosed in single quotes, e.g., 'A'. Double quotes are used for string literals.*

Q52.

What is the ASCII value of 'A'?

(A) 65 ✓

(B) 97

(C) 48

(D) 90

Explanation: *'A' has ASCII value 65. Lowercase 'a' is 97. '0' (digit zero) is 48.*

Q53.

What is the type of a character constant in C?

(A) char

(B) int ✓

(C) unsigned char

(D) short

Explanation: *In C (unlike C++), a character constant like 'A' has type int. This is a common source of confusion; sizeof('A') equals sizeof(int) in C.*

Q54.

What does the escape sequence '\n' represent?

(A) Null character

(B) Newline ✓

(C) Tab

(D) Backslash n

Explanation: *'\n' is the newline escape sequence (ASCII 10). It moves the cursor to the beginning of the next line.*

Q55.

What is the escape sequence for a tab character?

(A) \n

(B) \t ✓

(C) \r

(D) \b

Explanation: *'\t' represents a horizontal tab (ASCII 9). \n = newline, \r = carriage return, \b = backspace.*

Q56.

Which escape sequence represents the null character?

(A) \n

(B) \0 ✓

(C) \t

(D) \r

Explanation: *'\0' is the null character (ASCII 0). It is used as the string terminator in C.*

Q57.

What is the value of 'A' + 1 in C?

(A) 'B'

(B) 66

(C) Both A and B ✓

(D) 'A1'

Explanation: *'A' + 1 = 65 + 1 = 66, which is the ASCII value of 'B'. Both 'B' and 66 are correct representations of the result.*

Q58.

What is the output of printf("%c", 65);?

(A) 65

(B) A ✓

(C) a

(D) Error

Explanation: *%c prints the character corresponding to the ASCII value 65, which is 'A'.*

Q59.

Which of the following is a valid character constant?

(A) 'AB'

(B) '\n'

(C) '\0'

(D) Both B and C ✓

Explanation: *'\n' (newline) and '\0' (null) are valid single-character escape-sequence constants. 'AB' has two characters, which is a multi-character constant — implementation-defined and generally avoided.*

Q60.

What does '\\' represent in C?

(A) Two backslashes

(B) One backslash ✓

(C) Null character

(D) Invalid escape

Explanation: *'\\' is an escape sequence that represents a single backslash character. The first backslash escapes the second.*

Q61.

What is the escape sequence for a single quote?

(A) \' ✓

(B) \"

(C) \q

(D) \s

Explanation: *\' is the escape sequence for a single quote character. It is necessary inside character constants: char c = '\'';*

Q62.

What is the ASCII value of '0' (digit zero)?

(A) 0

(B) 30

(C) 48 ✓

(D) 52

Explanation: *The character '0' has ASCII value 48. This is different from the integer value 0 or the null character '\0' (ASCII 0).*

Q63.

What does '\a' represent?

(A) Alphabet

(B) Alert (bell) ✓

(C) Arrow

(D) ASCII

Explanation: *'\a' is the alert or bell escape sequence (ASCII 7). It causes the terminal to produce an audible beep.*

Q64.

Which of the following is the escape sequence for carriage return?

(A) \n

(B) \r ✓

(C) \t

(D) \c

Explanation: *'\r' is the carriage return escape sequence (ASCII 13). It moves the cursor to the beginning of the current line without advancing to the next line.*

Q65.

What is the output of: printf("%d", 'a' - 'A');?

(A) 0

(B) 32 ✓

(C) 1

(D) 65

Explanation: *'a' = 97, 'A' = 65. 97 - 65 = 32. The difference between lowercase and uppercase ASCII letters is always 32.*

Q66.

Error-finding: char c = 'AB'; — What is the issue?

(A) No error

(B) Multi-character constant: implementation-defined, not portable ✓

(C) Syntax error: cannot use two chars

(D) Runtime error

Explanation: *'AB' is a multi-character constant, which is allowed by the C standard but its value is implementation-defined. It is not portable and produces a compiler warning.*

Q67.

What does printf("%c", '\101'); print?

(A) 101

(B) A ✓

(C) e

(D) Error

Explanation: *'\101' is an octal escape sequence. Octal 101 = 64 + 1 = 65 decimal = 'A'. So printf prints A.*

Q68.

Which escape sequence represents a backspace?

(A) \b ✓

(B) \s

(C) \h

(D) \d

Explanation: *'\b' represents backspace (ASCII 8). It moves the cursor one position to the left.*

Q69.

What is the value of '9' - '0'?

(A) 9 ✓

(B) 57

(C) 0

(D) 48

Explanation: *'9' has ASCII 57, '0' has ASCII 48. 57 - 48 = 9. This trick converts a digit character to its numeric value.*

Q70.

What does the escape sequence '\x41' represent?

(A) x41

(B) A ✓

(C) 41

(D) Error

Explanation: *'\x41' is a hexadecimal escape sequence. 0x41 = 65 decimal = 'A' in ASCII.*

Q71.

Which of the following is a valid escape sequence in C?

(A) \p

(B) \v ✓

(C) \k

(D) \m

Explanation: *'\v' is a valid escape sequence representing the vertical tab character (ASCII 11). \p, \k, and \m are not standard escape sequences.*

Q72.

What is sizeof('A') in C?

(A) 1

(B) 2

(C) 4 ✓

(D) Depends on compiler

Explanation: *In C, character constants have type int. Therefore sizeof('A') = sizeof(int), which is typically 4 bytes. This differs from C++, where sizeof('A') = 1.*

Q73.

Which header defines character-testing functions like isalpha(), isdigit()?

(A) string.h

(B) ctype.h ✓

(C) stdio.h

(D) stdlib.h

Explanation: *ctype.h declares character classification functions: isalpha(), isdigit(), isupper(), islower(), isspace(), etc.*

Q74.

What is the output of: printf("%d", sizeof('z'));?

(A) 1

(B) 2

(C) 4 ✓

(D) 8

Explanation: *In C, 'z' is of type int. sizeof(int) is typically 4 on a 32-bit or 64-bit system.*

Q75.

What does '\f' represent?

(A) File feed

(B) Form feed ✓

(C) Function

(D) False

Explanation: *'\f' is the form feed escape sequence (ASCII 12). It advances the printer to the top of the next page.*

SECTION — STRING CONSTANTS

Q76.

How is a string constant (string literal) enclosed in C?

(A) Single quotes ' '

(B) Double quotes " " ✓

(C) Backticks ` `

(D) Curly braces { }

Explanation: *String literals in C are enclosed in double quotes, e.g., "Hello". Single quotes are for character constants.*

Q77.

What character is automatically appended at the end of every string constant in C?

(A) '\n'

(B) '\0' ✓

(C) ' '

(D) EOF

Explanation: *C automatically appends a null character '\0' (ASCII 0) at the end of every string literal, marking the end of the string.*

Q78.

How much memory does the string "Hello" occupy?

(A) 5 bytes

(B) 6 bytes ✓

(C) 4 bytes

(D) 10 bytes

Explanation: *"Hello" has 5 characters plus the null terminator '\0', totalling 6 bytes.*

Q79.

What is the difference between 'A' and "A" in C?

(A) No difference

(B) 'A' is a char constant (int type); "A" is a string with 'A' and '\0' ✓

(C) 'A' is a string; "A" is a char

(D) Both are strings

Explanation: *'A' is a character constant of type int with value 65. "A" is a string literal stored as {'A', '\0'} — an array of 2 chars.*

Q80.

What does strlen("Hello") return?

(A) 6

(B) 5 ✓

(C) 4

(D) 7

Explanation: *strlen() counts characters up to (but not including) the null terminator. "Hello" has 5 characters, so strlen returns 5.*

Q81.

Which of the following correctly declares a string constant?

(A) char s[] = 'Hello';

(B) char s[] = "Hello"; ✓

(C) char s = "Hello";

(D) string s = "Hello";

Explanation: *char s[] = "Hello"; is correct. The array size is automatically set to 6 (5 chars + null terminator). Single quotes are wrong for strings, and 'string' is not a C keyword.*

Q82.

What is a string literal stored in memory in C?

(A) A sequence of characters ending with EOF

(B) A null-terminated array of characters ✓

(C) A fixed-length array

(D) A pointer to a heap allocation

Explanation: *A string literal is stored as a null-terminated array of characters (char array). The null character '\0' marks the end.*

Q83.

What is the output of printf("%s", "Hello\0World");?

(A) Hello\0World

(B) HelloWorld

(C) Hello ✓

(D) World

Explanation: *printf with %s prints characters until it encounters '\0'. Since '\0' appears after 'Hello', only "Hello" is printed.*

Q84.

Two adjacent string literals in C are automatically:

(A) An error

(B) Treated as separate strings

(C) Concatenated by the compiler ✓

(D) Concatenated at runtime

Explanation: *Adjacent string literals are concatenated at compile time. For example, "Hello" " " "World" becomes "Hello World".*

Q85.

Is it legal to modify a string literal in C?

(A) Yes, always

(B) No, it is undefined behaviour ✓

(C) Yes, if stored in a char array

(D) Yes, but only the first character

Explanation: *String literals are typically stored in read-only memory. Modifying them is undefined behaviour. To modify, copy the literal into a char array first.*

Q86.

What does sizeof("Hello") return?

(A) 5

(B) 6 ✓

(C) 7

(D) 4

Explanation: *sizeof includes the null terminator. "Hello" = 5 characters + 1 null = 6 bytes.*

Q87.

Which function is used to copy one string to another?

(A) strcat()

(B) strcpy() ✓

(C) strcmp()

(D) strlen()

Explanation: *strcpy(dest, src) copies the string pointed to by src (including '\0') into the array pointed to by dest.*

Q88.

Which header must be included to use string functions like strlen, strcpy?

(A) stdio.h

(B) string.h ✓

(C) stdlib.h

(D) ctype.h

Explanation: *string.h declares string manipulation functions: strlen, strcpy, strcat, strcmp, strncpy, etc.*

Q89.

What is the output of: printf("%d", sizeof("C"));?

(A) 1

(B) 2 ✓

(C) 4

(D) 0

Explanation: *"C" is a string literal containing 'C' and '\0' — 2 bytes total. sizeof("C") = 2.*

Q90.

Which of the following is a wide string literal in C?

(A) L"Hello" ✓

(B) W"Hello"

(C) w"Hello"

(D) U"Hello"

Explanation: *A wide string literal is prefixed with L, e.g., L"Hello". Each character is of type wchar_t instead of char.*

Q91.

Error-finding: char *s = "Hello"; s[0] = 'h'; — What is the problem?

(A) No problem

(B) Undefined behaviour: modifying a string literal ✓

(C) Syntax error

(D) s[0] should be 'H'

Explanation: *String literals are stored in read-only memory. Attempting to modify s[0] causes undefined behaviour (often a segmentation fault at runtime).*

Q92.

What does strcmp("abc", "abc") return?

(A) 1

(B) -1

(C) 0 ✓

(D) True

Explanation: *strcmp() returns 0 when both strings are equal, a negative value if the first is less, and a positive value if the first is greater.*

Q93.

What is the output of: printf("%c", "Hello"[1]);?

(A) H

(B) e ✓

(C) l

(D) Error

Explanation: *"Hello"[1] accesses the character at index 1 (0-based). "Hello"[0]='H', [1]='e'. So %c prints 'e'.*

Q94.

Which escape sequence within a string outputs a double quote?

(A) \" ✓

(B) \'

(C) \q

(D) \d

Explanation: *\" is the escape sequence for a double quote inside a string literal. Without it, the quote would terminate the string.*

Q95.

What is a string constant in C technically stored as?

(A) A linked list of characters

(B) An array of char ending with '\0' ✓

(C) A fixed 256-byte buffer

(D) A std::string object

Explanation: *A string constant is stored as a char array with a null terminator ('\0') at the end. There is no std::string in C (that is C++).*

Q96.

Which of the following concatenates two strings in C?

(A) s1 + s2

(B) strcat(s1, s2) ✓

(C) concat(s1, s2)

(D) append(s1, s2)

Explanation: *strcat(s1, s2) appends s2 to the end of s1. The + operator does not work for string concatenation in C.*

Q97.

What is the value of *"Hello"?

(A) 'H' ✓

(B) 'e'

(C) The whole string

(D) NULL

Explanation: *A string literal decays to a pointer to its first character. Dereferencing it (*"Hello") gives the first character 'H'.*

Q98.

Which of the following defines an unmodifiable string?

(A) char s[] = "Hello";

(B) const char *s = "Hello"; ✓

(C) char *s = "Hello";

(D) Both A and B

Explanation: *const char *s = "Hello"; makes the string unmodifiable through the pointer. char s[] = "Hello"; allows modification because it copies the literal into a local array.*

Q99.

What is the output of printf("%s", "");?

(A) (nothing) ✓

(B) NULL

(C) \0

(D) Error

Explanation: *An empty string "" contains only the null terminator. printf with %s prints nothing for an empty string.*

Q100.

Error-finding: char s[5] = "Hello"; — What is the issue?

(A) No issue

(B) Array is too small: needs 6 bytes for 'Hello' + '\0' ✓

(C) String literals cannot initialise arrays

(D) char cannot hold 'Hello'

Explanation: *"Hello" requires 6 bytes (5 chars + null terminator). char s[5] can only hold 5 bytes, so there is no room for '\0'. This is a buffer overflow.*

SECTION — SYMBOLIC CONSTANTS

Q101.

Which preprocessor directive is used to define a symbolic constant?

(A) #include

(B) #define ✓

(C) #ifdef

(D) #pragma

Explanation: *#define is used to define symbolic constants (macros). Example: #define PI 3.14159. The preprocessor replaces every occurrence of PI with 3.14159 before compilation.*

Q102.

Which keyword is used to define a typed constant in C?

(A) define

(B) constant

(C) const ✓

(D) static

Explanation: *The const keyword declares a variable as a typed, read-only constant. Example: const float PI = 3.14159f; Unlike #define, const has a type and scope.*

Q103.

What is the convention for naming symbolic constants defined with #define?

(A) camelCase

(B) ALL_UPPERCASE ✓

(C) PascalCase

(D) lowercase_with_underscores

Explanation: *By convention, #define macro names are written in ALL_UPPERCASE (with underscores for words), e.g., MAX_SIZE, PI, BUFFER_LENGTH. This distinguishes them from variables.*

Q104.

What is the advantage of using symbolic constants over magic numbers?

(A) Faster execution

(B) Easier to read, maintain, and change the code ✓

(C) Reduced memory usage

(D) Automatic type checking

Explanation: *Symbolic constants improve readability (PI is clearer than 3.14159) and maintainability — changing the constant in one place updates it everywhere.*

Q105.

What does #define MAX 100 do?

(A) Allocates memory for MAX

(B) Creates a variable MAX

(C) Tells the preprocessor to replace MAX with 100 in source code ✓

(D) Declares MAX as a function

Explanation: *#define MAX 100 creates a preprocessor macro. Before compilation, every occurrence of MAX in the source code is replaced by the text 100.*

Q106.

What is the difference between #define PI 3.14 and const float PI = 3.14f;?

(A) No difference

(B) #define has no type; const PI has type float with scope and type-checking ✓

(C) #define is faster; const is slower

(D) const cannot store decimals

Explanation: *#define performs text substitution without type information. const gives the constant a type (float), scope, and allows type-checking by the compiler.*

Q107.

Which of the following correctly defines a symbolic constant for the maximum array size?

(A) #define MAX_SIZE = 100;

(B) #define MAX_SIZE 100 ✓

(C) define MAX_SIZE 100;

(D) #DEFINE MAX_SIZE 100

Explanation: *#define MAX_SIZE 100 is correct. There is no = or semicolon in a #define. The directive is case-sensitive: #DEFINE is invalid.*

Q108.

Can a const variable be used as an array size in standard C (C89/C90)?

(A) Yes, always

(B) No, array sizes must be integer constants or #define macros in C89 ✓

(C) Yes, but only for local arrays

(D) Yes, if the value is known at compile time

Explanation: *In C89/C90, array dimensions must be integer constant expressions. const variables are not considered constant expressions, so they cannot be used. C99 introduced VLAs as an alternative.*

Q109.

Error-finding: #define LIMIT = 50; — What is wrong?

(A) No error

(B) The = sign is wrong; #define uses a space, not = ✓

(C) Missing data type

(D) Semicolons are required

Explanation: *#define uses a space, not an equals sign. Also, the trailing semicolon becomes part of the replacement text, causing errors wherever LIMIT is used (e.g., if (x < LIMIT;)). Correct: #define LIMIT 50*

Q110.

Which directive removes a previously defined macro?

(A) #remove

(B) #undef ✓

(C) #delete

(D) #clear

Explanation: *#undef removes a macro definition, e.g., #undef PI. After this, PI is no longer defined and using it causes a compile error.*

Q111.

What is a 'magic number' in programming?

(A) A constant defined with #define

(B) An unnamed numeric literal used directly in code ✓

(C) A number generated by a random-number algorithm

(D) A value stored in a const variable

Explanation: *A 'magic number' is a hard-coded numeric literal (like 100 or 3.14) embedded directly in code without explanation. Symbolic constants replace magic numbers to improve readability.*

Q112.

Which of the following defines a constant using the enum mechanism?

(A) #define RED 0

(B) const int RED = 0;

(C) enum Color { RED = 0 };

(D) All of the above define RED as 0 ✓

Explanation: *All three methods define RED with value 0. enum is particularly useful for groups of related integer constants, providing named values within a type.*

Q113.

What is the scope of a #define macro?

(A) Local to the function where it is defined

(B) Local to the block where it is defined

(C) From the point of definition to the end of the translation unit (or #undef) ✓

(D) Global across all files

Explanation: *#define macros have file scope — they apply from the point of definition until the end of the file or until #undef is encountered. They are not block-scoped.*

Q114.

Which standard header defines constants like INT_MAX and INT_MIN?

(A) stdlib.h

(B) limits.h ✓

(C) float.h

(D) stddef.h

Explanation: *limits.h defines integer type limits: INT_MAX, INT_MIN, LONG_MAX, CHAR_MAX, etc. float.h defines floating-point limits.*

Q115.

What is the value of INT_MAX on a 32-bit system?

(A) 32767

(B) 65535

(C) 2147483647 ✓

(D) 4294967295

Explanation: *INT_MAX = 2^31 - 1 = 2147483647 for a 32-bit signed int.*

Q116.

Can #define macros take parameters?

(A) No, macros are always simple replacements

(B) Yes, they are called function-like macros ✓

(C) Yes, but only one parameter

(D) No, use inline functions instead

Explanation: *Macros can take parameters: #define SQUARE(x) ((x)*(x)). These are function-like macros. The extra parentheses are important to prevent operator-precedence bugs.*

Q117.

Error-finding: What is wrong? #define AREA(r) 3.14 * r * r — printf("%f", AREA(2+3));

(A) No error

(B) Result is 3.14 * 2+3 * 2+3 = wrong due to missing parentheses around r ✓

(C) Macro cannot be used in printf

(D) 3.14 should be PI

Explanation: *AREA(2+3) expands to 3.14 * 2+3 * 2+3, which is evaluated as (3.14*2) + (3*2) + 3 = 6.28+6+3 = 15.28 instead of 3.14*25=78.5. Always parenthesise macro parameters: #define AREA(r) (3.14*(r)*(r))*

Q118.

Which of the following declares a constant pointer to an integer?

(A) const int *p;

(B) int * const p; ✓

(C) int const *p;

(D) const int const *p;

Explanation: *int * const p; declares a constant pointer to int (the pointer itself cannot change). const int *p; declares a pointer to a constant int (the value cannot change through p).*

Q119.

What is the output? #define DOUBLE(x) x+x — printf("%d", DOUBLE(3)*2);

(A) 12

(B) 9 ✓

(C) 18

(D) 6

Explanation: *DOUBLE(3)*2 expands to 3+3*2 = 3+6 = 9 (operator precedence: * before +). Correct macro: #define DOUBLE(x) ((x)+(x)).*

Q120.

Which of the following is the best practice for defining a constant in C?

(A) Using #define for all constants

(B) Using const for typed constants and #define for macros that need type-independence ✓

(C) Using global variables

(D) Using static local variables

Explanation: *Best practice: use const for typed, scoped constants (benefits from type-checking and debugger support). Use #define for macros, include guards, and where a constant expression is required (e.g., C89 array sizes).*

CONSOLIDATED ANSWER KEY

Q1: (A)	Q2: (B)	Q3: (C)	Q4: (C)	Q5: (C)
Q6: (C)	Q7: (B)	Q8: (A)	Q9: (D)	Q10: (C)
Q11: (B)	Q12: (D)	Q13: (C)	Q14: (B)	Q15: (D)
Q16: (B)	Q17: (C)	Q18: (C)	Q19: (B)	Q20: (C)
Q21: (A)	Q22: (B)	Q23: (B)	Q24: (C)	Q25: (B)
Q26: (D)	Q27: (B)	Q28: (B)	Q29: (C)	Q30: (D)
Q31: (C)	Q32: (B)	Q33: (C)	Q34: (B)	Q35: (A)
Q36: (D)	Q37: (B)	Q38: (B)	Q39: (A)	Q40: (A)
Q41: (C)	Q42: (B)	Q43: (B)	Q44: (B)	Q45: (B)
Q46: (C)	Q47: (B)	Q48: (B)	Q49: (B)	Q50: (B)
Q51: (B)	Q52: (A)	Q53: (B)	Q54: (B)	Q55: (B)
Q56: (B)	Q57: (C)	Q58: (B)	Q59: (D)	Q60: (B)
Q61: (A)	Q62: (C)	Q63: (B)	Q64: (B)	Q65: (B)
Q66: (B)	Q67: (B)	Q68: (A)	Q69: (A)	Q70: (B)
Q71: (B)	Q72: (C)	Q73: (B)	Q74: (C)	Q75: (B)
Q76: (B)	Q77: (B)	Q78: (B)	Q79: (B)	Q80: (B)
Q81: (B)	Q82: (B)	Q83: (C)	Q84: (C)	Q85: (B)
Q86: (B)	Q87: (B)	Q88: (B)	Q89: (B)	Q90: (A)
Q91: (B)	Q92: (C)	Q93: (B)	Q94: (A)	Q95: (B)
Q96: (B)	Q97: (A)	Q98: (B)	Q99: (A)	Q100: (B)
Q101: (B)	Q102: (C)	Q103: (B)	Q104: (B)	Q105: (C)
Q106: (B)	Q107: (B)	Q108: (B)	Q109: (B)	Q110: (B)
Q111: (B)	Q112: (D)	Q113: (C)	Q114: (B)	Q115: (C)
Q116: (B)	Q117: (B)	Q118: (B)	Q119: (B)	Q120: (B)

Integer Constants: Q1→(A) Q2→(B) Q3→(C) Q4→(C) Q5→(C) Q6→(C) Q7→(B) Q8→(A) Q9→(D) Q10→(C) Q11→(B) Q12→(D) Q13→(C) Q14→(B) Q15→(D) Q16→(B) Q17→(C) Q18→(C) Q19→(B) Q20→(C) Q21→(A) Q22→(B) Q23→(B) Q24→(C) Q25→(B)

Floating-Point Constants: Q26→(D) Q27→(B) Q28→(B) Q29→(C) Q30→(D) Q31→(C) Q32→(B) Q33→(C) Q34→(B) Q35→(A) Q36→(D) Q37→(B) Q38→(B) Q39→(A) Q40→(A) Q41→(C) Q42→(B) Q43→(B) Q44→(B) Q45→(B) Q46→(C) Q47→(B) Q48→(B) Q49→(B) Q50→(B)

Character Constants: Q51→(B) Q52→(A) Q53→(B) Q54→(B) Q55→(B) Q56→(B) Q57→(C) Q58→(B) Q59→(D) Q60→(B) Q61→(A) Q62→(C) Q63→(B) Q64→(B) Q65→(B) Q66→(B) Q67→(B) Q68→(A) Q69→(A) Q70→(B) Q71→(B) Q72→(C) Q73→(B) Q74→(C) Q75→(B)

String Constants: Q76→(B) Q77→(B) Q78→(B) Q79→(B) Q80→(B) Q81→(B) Q82→(B) Q83→(C) Q84→(C) Q85→(B) Q86→(B) Q87→(B) Q88→(B) Q89→(B) Q90→(A) Q91→(B) Q92→(C) Q93→(B) Q94→(A) Q95→(B) Q96→(B) Q97→(A) Q98→(B) Q99→(A) Q100→(B)

Symbolic Constants: Q101→(B) Q102→(C) Q103→(B) Q104→(B) Q105→(C) Q106→(B) Q107→(B) Q108→(B) Q109→(B) Q110→(B) Q111→(B) Q112→(D) Q113→(C) Q114→(B) Q115→(C) Q116→(B) Q117→(B) Q118→(B) Q119→(B) Q120→(B)

QUICK REVISION NOTES — CONSTANTS IN C

Integer Constants	• Forms: Decimal (123), Octal (0755), Hexadecimal (0xFF) • Suffixes: U/u = unsigned, L/l = long, UL = unsigned long, LL = long long • No spaces, commas, or underscores in C89/90 • Invalid: 09 (octal cannot contain 8 or 9)
Floating-Point Constants	• Default type is double (e.g., 3.14 is double) • Suffix f/F = float; l/L = long double • Scientific notation: 1.5e3 = 1500.0 • Exponent must have at least one digit: 1e is invalid
Character Constants	• Enclosed in single quotes: 'A', '\n', '\0' • Type is int in C (not char!) • Common escape sequences: \n \t \r \b \0 \\ \' \" • 'A' = 65, 'a' = 97, '0' = 48 (ASCII values)
String Constants	• Enclosed in double quotes: "Hello" • Automatically null-terminated ('\0' appended) • sizeof("Hello") = 6; strlen("Hello") = 5 • String literals are read-only — modifying them is UB • Adjacent literals are concatenated at compile time
Symbolic Constants	• #define PI 3.14159 — no type, no scope, text substitution • const float PI = 3.14f; — typed, scoped, debuggable • #define has no = and no semicolon at end • #undef removes a macro definition • Always parenthesise macro parameters to avoid precedence bugs

Best of luck for your OAVS Computer Teacher (TGT) Examination!